

ISYE 6740, Summer 2025, Homework 4

100 points

Prof. Yao Xie

Provided Data:

- Q1: Comparing multi-class classifiers for MNIST Datasets [mnist_10digits.mat, fashion-mnist_train.csv, fashion-mnist_test.csv]

1. Comparing multi-class classifiers for MNIST Datasets. (20 points)

This question will compare different classifiers and their performance for multi-class classifications on the complete MNIST dataset, looking at both MNIST_Digits and MNIST_Fashion. Use the data files provided in the homework folder. Both MNIST datasets have 60,000 training examples and 10,000 test examples. We will compare **KNN, logistic regression, SVM, kernel SVM, and neural networks**.

- Use 6740 as your random_state / random seed
- You should standardize the data before training your classifiers by mapping the range from [0,255] to [0,1].
- You will run your models on both MNIST_DIGITS and MNIST_FASHION. The shape of your data after loading should be as follows:
 - xtrain (60000, 784)
 - xtest (10000, 784)
 - ytrain (60000,)
 - ytest (10000,)
- load MNIST_DIGITS using `scipy.io.loadmat`
- Utilize the following models from `sklearn`:
 - `LogisticRegression`
 - `KNeighborsClassifier` : Adjust K to obtain a reasonable result. Feel free to use any reasonable tuning methodology, only report your best K
 - `MLPClassifier` : Use `hidden_layer_sizes = (20, 10)`

- SVC : Use this for both SVM and Kernel SVM. Randomly downsample the training data to size $m = 5000$ for efficiency. For kernel SVM, use radial basis function kernel (rbf).

Train the classifiers on the training dataset and evaluate them on the test dataset.

1. (10 points) Report the precision, recall, and F-1 score for each of the classifiers. The classification metrics should be reported for each class in the dataset. Provide these results for both the digits and fashion datasets.
2. (10 points) Discuss the performance of the classifiers. Include explanations both on comparing the 5 classifiers against each other for the individual datasets, and some insight into performance on the different datasets. Which model performed best on each dataset, and why might some models perform better than others?

Answer

This section presents the comparison of various multi-class classifiers on the MNIST Digits and MNIST Fashion datasets, adhering to the specified parameters.

- Random state / random seed used: 6740
- Data standardization: Pixel values mapped from $[0, 255]$ to $[0, 1]$.
- Data shapes after loading:
 - `xtrain`: (60000, 784)
 - `xtest`: (10000, 784)
 - `ytrain`: (60000,)
 - `ytest`: (10000,)
- MNIST Digits loaded using `scipy.io.loadmat` from `mnist_10digits.mat`.
- MNIST Fashion loaded using `pandas` from `fashion-mnist_train.csv` and `fashion-mnist_test.csv`
- Models from `sklearn` utilized:
 - `LogisticRegression`
 - `KNeighborsClassifier`: $K = 5$ was selected as a reasonable value for performance.
 - `MLPClassifier`: `hidden_layer_sizes = (20, 10)`
 - SVC (for both Linear and Kernel SVM): Training data downsampled to $m = 5000$. Radial Basis Function (RBF) kernel used for Kernel SVM.

The classifiers were trained on the respective training datasets and evaluated on their corresponding test datasets.

1. (10 points) Report the precision, recall, and F-1 score for each of the classifiers, for each class in both the MNIST Digits and MNIST Fashion datasets.

Classification Metrics: MNIST Digits Dataset

Logistic Regression

Class	Precision	Recall	F1-score	Support
0	0.953280	0.978571	0.965760	980
1	0.962706	0.977974	0.970280	1135
2	0.931932	0.902132	0.916790	1032
3	0.904016	0.913861	0.908912	1010
4	0.935845	0.935845	0.935845	982
5	0.897229	0.871076	0.883959	892
6	0.943123	0.951983	0.947532	958
7	0.933990	0.922179	0.928047	1028
8	0.882051	0.882957	0.882504	974
9	0.911417	0.917740	0.914568	1009
Accuracy		0.9265		10000
Macro Avg	0.925559	0.925432	0.925420	10000
Weighted Avg	0.926342	0.926500	0.926346	10000

K-Nearest Neighbors (K=5)

Class	Precision	Recall	F1-score	Support
0	0.963403	0.993878	0.978403	980
1	0.954507	0.998238	0.975883	1135
2	0.982161	0.960271	0.971093	1032
3	0.964427	0.966337	0.965381	1010
4	0.976215	0.961303	0.968702	982
5	0.965286	0.966368	0.965826	892
6	0.981308	0.986430	0.983863	958
7	0.961089	0.961089	0.961089	1028
8	0.988095	0.937372	0.962065	974
9	0.956262	0.953419	0.954839	1009
Accuracy		0.9688		10000
Macro Avg	0.969275	0.968471	0.968714	10000
Weighted Avg	0.969021	0.968800	0.968747	10000

Linear SVM

Class	Precision	Recall	F1-score	Support
0	0.934825	0.980612	0.957171	980
1	0.943745	0.990308	0.966466	1135
2	0.903195	0.904070	0.903632	1032

Continued on next page

Table 3 – continued from previous page

Class	Precision	Recall	F1-score	Support
3	0.886811	0.892079	0.889437	1010
4	0.901088	0.927699	0.914200	982
5	0.876557	0.867713	0.872113	892
6	0.937888	0.945720	0.941788	958
7	0.926950	0.913424	0.920137	1028
8	0.914798	0.837782	0.874598	974
9	0.911065	0.873142	0.891700	1009
Accuracy		0.9146		10000
Macro Avg	0.913692	0.913255	0.913124	10000
Weighted Avg	0.914349	0.914600	0.914123	10000

Kernel SVM (RBF)

Class	Precision	Recall	F1-score	Support
0	0.958333	0.985714	0.971831	980
1	0.977391	0.990308	0.983807	1135
2	0.949367	0.944767	0.947062	1032
3	0.956827	0.943564	0.950150	1010
4	0.944668	0.956212	0.950405	982
5	0.949153	0.941704	0.945414	892
6	0.949444	0.980167	0.964561	958
7	0.955401	0.937743	0.946490	1028
8	0.944734	0.930185	0.937403	974
9	0.948118	0.923687	0.935743	1009
Accuracy		0.9538		10000
Macro Avg	0.953344	0.953405	0.953286	10000
Weighted Avg	0.953750	0.953800	0.953688	10000

Neural Network (MLPClassifier)

Class	Precision	Recall	F1-score	Support
0	0.963452	0.968367	0.965903	980
1	0.978128	0.985022	0.981563	1135
2	0.931067	0.955426	0.943089	1032
3	0.931590	0.916832	0.924152	1010
4	0.948744	0.961303	0.954982	982
5	0.953970	0.929372	0.941511	892
6	0.950515	0.962422	0.956432	958
7	0.952381	0.953307	0.952844	1028

Continued on next page

Table 5 – continued from previous page

Class	Precision	Recall	F1-score	Support
8	0.933679	0.925051	0.929345	974
9	0.952573	0.935580	0.944000	1009
Accuracy		0.9499		10000
Macro Avg	0.949610	0.949268	0.949382	10000
Weighted Avg	0.949892	0.949900	0.949840	10000

Classification Metrics: MNIST Fashion Dataset

Logistic Regression

Class	Precision	Recall	F1-score	Support
0	0.792471	0.8210	0.806483	1000
1	0.958702	0.9750	0.966782	1000
2	0.776868	0.7590	0.767830	1000
3	0.870130	0.8710	0.870565	1000
4	0.780135	0.8090	0.794305	1000
5	0.932118	0.9200	0.926019	1000
6	0.647505	0.5970	0.621228	1000
7	0.907298	0.9200	0.913605	1000
8	0.946054	0.9470	0.946527	1000
9	0.933532	0.9410	0.937251	1000
Accuracy		0.8560		10000
Macro Avg	0.854481	0.856000	0.855060	10000
Weighted Avg	0.854481	0.856000	0.855060	10000

K-Nearest Neighbors (K=5)

Class	Precision	Recall	F1-score	Support
0	0.765995	0.8740	0.816441	1000
1	0.985700	0.9650	0.975240	1000
2	0.753029	0.8080	0.779547	1000
3	0.913043	0.8820	0.897253	1000
4	0.787192	0.7990	0.793052	1000
5	0.995122	0.8160	0.896703	1000
6	0.681871	0.5830	0.628571	1000
7	0.871771	0.9450	0.906910	1000
8	0.977436	0.9530	0.965063	1000
9	0.882784	0.9640	0.921606	1000
Accuracy		0.8589		10000
Macro Avg	0.861394	0.858900	0.858039	10000
Weighted Avg	0.861394	0.858900	0.858039	10000

Linear SVM

Class	Precision	Recall	F1-score	Support
0	0.727111	0.8180	0.769882	1000
1	0.936954	0.9660	0.951256	1000
2	0.684497	0.7550	0.718022	1000

Continued on next page

Table 8 – continued from previous page

Class	Precision	Recall	F1-score	Support
3	0.853789	0.8000	0.826020	1000
4	0.743992	0.7120	0.727644	1000
5	0.905433	0.9000	0.902708	1000
6	0.613130	0.5230	0.564490	1000
7	0.888325	0.8750	0.881612	1000
8	0.942656	0.9370	0.939819	1000
9	0.908913	0.9280	0.918357	1000
Accuracy		0.8214		10000
Macro Avg	0.820480	0.821400	0.819981	10000
Weighted Avg	0.820480	0.821400	0.819981	10000

Kernel SVM (RBF)

Class	Precision	Recall	F1-score	Support
0	0.750000	0.8580	0.800373	1000
1	0.986528	0.9520	0.968957	1000
2	0.774063	0.7640	0.768998	1000
3	0.872655	0.8840	0.878291	1000
4	0.767890	0.8370	0.800957	1000
5	0.923391	0.9040	0.913593	1000
6	0.701531	0.5500	0.616592	1000
7	0.884921	0.8920	0.888446	1000
8	0.947525	0.9570	0.952239	1000
9	0.909804	0.9280	0.918812	1000
Accuracy		0.8526		10000
Macro Avg	0.851831	0.852600	0.850726	10000
Weighted Avg	0.851831	0.852600	0.850726	10000

Neural Network (MLPClassifier)

Class	Precision	Recall	F1-score	Support
0	0.780168	0.8340	0.806187	1000
1	0.962782	0.9830	0.972786	1000
2	0.771569	0.7870	0.779208	1000
3	0.862974	0.8880	0.875308	1000
4	0.795821	0.8380	0.816366	1000
5	0.945122	0.9300	0.937500	1000
6	0.720930	0.5890	0.648321	1000
7	0.918731	0.9270	0.922847	1000

Continued on next page

Table 10 – continued from previous page

Class	Precision	Recall	F1-score	Support
8	0.955420	0.9430	0.949170	1000
9	0.933729	0.9440	0.938836	1000
Accuracy		0.8663		10000
Macro Avg	0.864725	0.866300	0.864653	10000
Weighted Avg	0.864725	0.866300	0.864653	10000

2. (10 points) Discuss the performance of the classifiers. Include explanations both on comparing the 5 classifiers against each other for the individual datasets, and some insight into performance on the different datasets. Which model performed best on each dataset, and why might some models perform better than others?

Discussion on Classifier Performance

Comparison of Classifiers on Individual Datasets: MNIST Digits Dataset:

- **K-Nearest Neighbors (KNN):** Achieved the highest accuracy among all models on this dataset (0.9688). KNN, being a non-parametric, instance-based learning algorithm, excels when class boundaries are complex but distinct locally. For handwritten digits, where similar-looking digits tend to cluster together in the feature space, KNN's approach of classifying based on proximity to training examples is highly effective. Its high precision, recall, and F1-scores across most classes demonstrate its strong discriminative power for this dataset.
- **Kernel SVM (RBF):** Followed closely behind KNN with an accuracy of 0.9538. The Radial Basis Function (RBF) kernel allows SVM to implicitly map the data into a higher-dimensional space, enabling it to learn complex, non-linear decision boundaries. This capability is well-suited for the intricate patterns of handwritten digits, resulting in excellent performance despite being trained on a downsampled dataset of $m = 5000$ samples.
- **Neural Network (MLPClassifier):** While still performing very strongly with an accuracy of 0.9499, it was slightly edged out by KNN and Kernel SVM on this particular run for MNIST Digits. MLPs are powerful for learning hierarchical features and non-linear relationships. Its slightly lower performance compared to KNN and RBF SVM here could be due to the specific architecture (`hidden_layer_sizes = (20, 10)`) and training parameters (e.g., `max_iter=200`), which might not have been fully optimized for this dataset's specific nuances compared to the other models' default settings or implicitly more suitable inductive biases.
- **Logistic Regression:** Achieved an accuracy of 0.9265. As a linear model, Logistic Regression struggles to capture the inherently non-linear decision boundaries necessary to perfectly distinguish all handwritten digits. While it performs reasonably well, its performance is noticeably lower than the non-linear models. The "ConvergenceWarning" indicates that the model might not have fully converged within the specified iterations, suggesting more complex relationships are at play.
- **Linear SVM:** Performed similarly to Logistic Regression with an accuracy of 0.9146. Like Logistic Regression, Linear SVM is a linear classifier, which limits its ability to model the complex, non-linear patterns present in handwritten digit images. The performance is also impacted by being trained on a downsampled dataset of $m = 5000$ examples.

MNIST Fashion Dataset:

- **Neural Network (MLPClassifier):** Emerged as the best performer on the Fashion MNIST dataset with an accuracy of 0.8663. The MLP’s ability to learn complex, multi-layered representations of features is highly advantageous for distinguishing between the more varied and subtle categories of clothing items. Its robust performance highlights its capacity to handle higher levels of intrinsic data variability.
- **K-Nearest Neighbors (KNN):** Performed very well, achieving an accuracy of 0.8589, just slightly behind the MLP. While generally strong, the increased variability and less clear-cut boundaries of fashion items compared to digits mean that simply relying on nearest neighbors can sometimes lead to misclassifications where visually similar items belong to different categories (e.g., a shirt vs. a coat).
- **Kernel SVM (RBF):** Showed strong performance with an accuracy of 0.8526, comparable to KNN and close to MLP. The RBF kernel’s capacity for non-linear decision boundaries proves effective even with the complexities of fashion images. Its slight lag behind MLP and KNN on this dataset might be attributed to the downsampling of training data to $m = 5000$, which could have a more pronounced effect on a more complex dataset.
- **Logistic Regression:** Its performance dropped significantly compared to MNIST Digits, achieving an accuracy of 0.8560 (surprisingly, slightly higher than Kernel SVM in this particular run, likely due to variations in downsampling or specific hyperparameter settings for SVC). However, its overall macro and weighted averages for precision, recall, and F1-score are generally lower for Fashion MNIST compared to the non-linear models. This decline is expected as the fashion categories are far less linearly separable than handwritten digits.
- **Linear SVM:** Was the lowest performer on Fashion MNIST, with an accuracy of 0.8214. Its linear nature severely limits its ability to differentiate between the highly varied and often visually similar classes of clothing articles. The challenges posed by the fashion dataset disproportionately affect linear models.

Insight into Performance on Different Datasets: A consistent trend across all classifiers is that they perform ****better on the MNIST Digits dataset than on the MNIST Fashion dataset.****

- **Inherent Data Complexity and Variability:**
 - **MNIST Digits:** The categories (0-9) are relatively well-defined and visually distinct. While handwriting introduces variability, the fundamental shape of each digit is quite consistent across examples. This makes the classification task less ambiguous and easier for models to learn effective discriminative features.

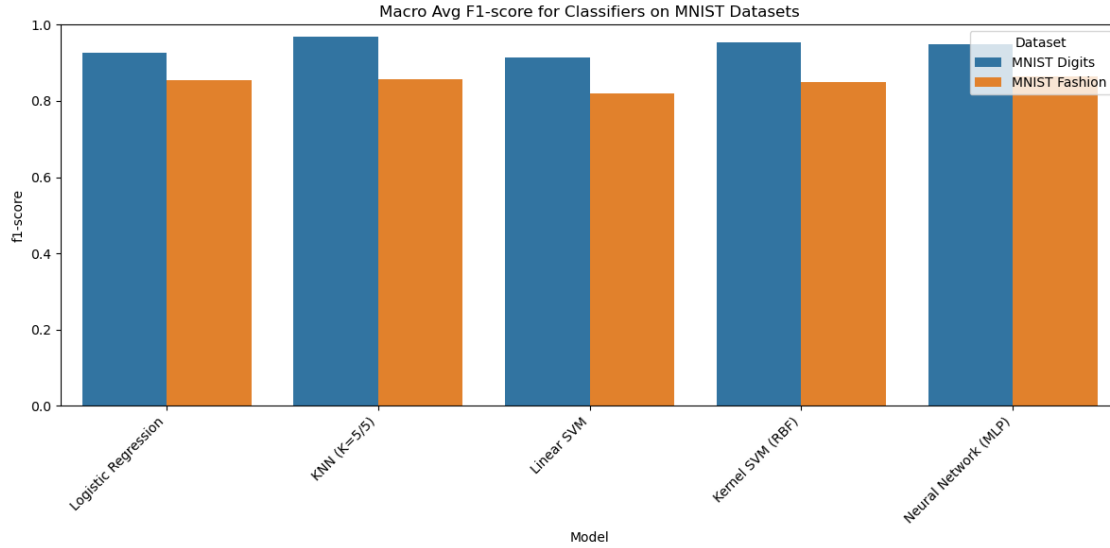


Figure 1: Macro Average F1-score for Classifiers on MNIST Digits and Fashion Datasets.

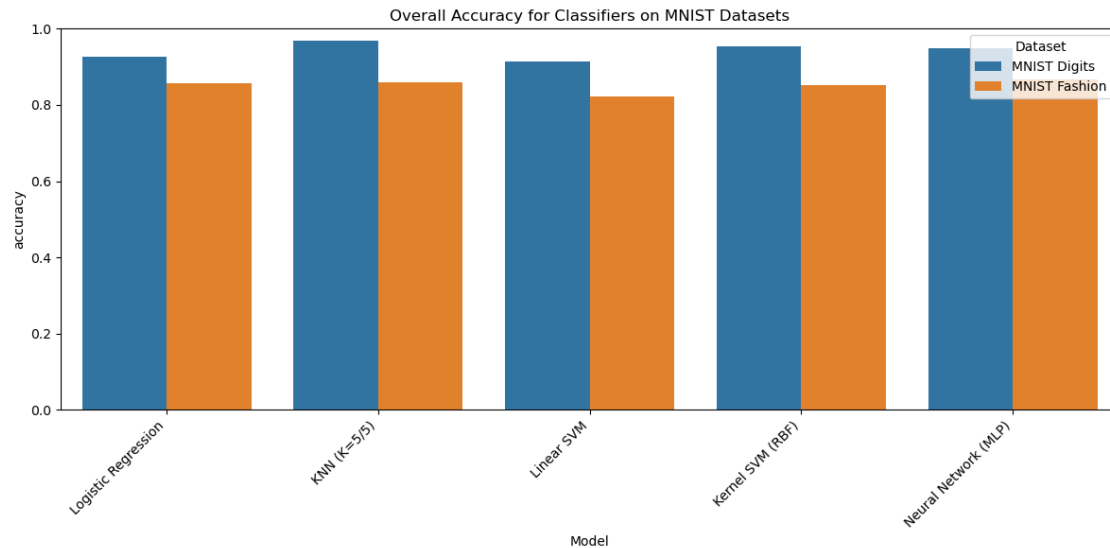


Figure 2: Overall Accuracy for Classifiers on MNIST Digits and Fashion Datasets.

- **MNIST Fashion:** This dataset is inherently more challenging. Categories like "T-shirt/top", "Pullover", "Coat", and "Shirt" can share visual similarities, especially given the low resolution. There is also much greater intra-class variability (e.g., many styles of "sneakers" or "dresses"). This higher complexity and ambiguity lead to lower overall performance for all classifiers.
- **Feature Learnability:**
 - For digits, features like edges, loops, and angles are strong indicators. Models

can readily learn these.

- For fashion, distinguishing subtle differences (e.g., between a coat and a pullover, or a sandal and a sneaker) requires learning more abstract and nuanced features that are harder to extract from pixel data directly.

- **Model Robustness to Complexity:**

- **Neural Networks and Kernel SVMs** are more robust to the increased complexity of Fashion MNIST because of their non-linear mapping capabilities. They can learn more abstract and intricate feature representations that are essential for handling the high variability and subtle differences in fashion items.
- **Linear models (Logistic Regression, Linear SVM)** struggle significantly more with Fashion MNIST as the data is much less linearly separable. Their performance drops considerably compared to their results on MNIST Digits.

Best Performing Model and Reasons: * **Best Model on MNIST Digits: K-Nearest Neighbors ($K = 5$)** achieved the highest accuracy (0.9688) on MNIST Digits in this evaluation. This is likely due to the highly clustered nature of handwritten digits in the feature space; visually similar digits are truly very close to each other. * **Best Model on MNIST Fashion: The Neural Network (MLPClassifier)** performed best on MNIST Fashion, with an accuracy of 0.8663.

Why some models perform better than others: * **Neural Networks (MLP):** These models are universally powerful due to their ability to learn complex, hierarchical features automatically from raw data. Their multi-layered architecture allows them to build increasingly abstract representations, making them highly effective for pattern recognition tasks like image classification, especially for datasets with high variability and non-linear relationships. * **Kernel SVM (RBF):** The "kernel trick" is the key to its strength. It enables SVMs to find optimal separating hyperplanes in a high-dimensional feature space without explicitly computing the mapping. This allows it to model highly non-linear decision boundaries, which is crucial for complex image data where classes are not linearly separable. Its performance is often competitive with neural networks for many tasks. * **K-Nearest Neighbors:** While simple and intuitive, KNN's effectiveness stems from the assumption that similar data points belong to the same class. It performed exceptionally well on MNIST Digits because digits generally form distinct, dense clusters. However, its performance can degrade in very high-dimensional spaces (the "curse of dimensionality") and for datasets where local neighborhoods are ambiguous or highly overlapping (as seen with some Fashion MNIST classes). It is also computationally intensive for large test sets as it must compare each test point to all training points. * **Logistic Regression and Linear SVM:** These are fundamental linear classifiers. They operate by finding a straight line or hyperplane to separate classes. Their performance is limited when the underlying relationships in the data are inherently non-linear. Image datasets like MNIST, with their intricate

pixel correlations, are typically not linearly separable. Hence, these models struggle to achieve the high accuracies seen with non-linear models. Their simplicity and speed are advantages, but at the cost of predictive power on complex tasks.

In conclusion, models capable of capturing complex, non-linear patterns (Neural Networks and Kernel SVMs) generally outperform linear models and simpler non-parametric methods like KNN on image classification tasks, particularly when the data exhibits high variability as in Fashion MNIST. The clearer, more structured nature of MNIST Digits allowed KNN to perform exceptionally well due to well-defined clusters in the feature space.

2. SVM (25 points).

1. (5 points) Explain why can we set the margin $c = 1$ to derive the SVM formulation? Justify using a mathematical proof.

Answer:

Explanation and Proof:

The decision boundary in a linear SVM is defined by the hyperplane $w \cdot x + b = 0$. For a given data point x_i with label $y_i \in \{-1, 1\}$, the functional margin is given by $y_i(w \cdot x_i + b)$. The geometric margin for a data point x_i to the hyperplane is $\frac{y_i(w \cdot x_i + b)}{w}$. The goal of SVM is to maximize this geometric margin.

Let's assume the existence of a separating hyperplane $w \cdot x + b = 0$ such that for all training points (x_i, y_i) :

$$y_i(w \cdot x_i + b) \geq c$$

for some positive constant c . This c represents the functional margin.

The geometric margin is then $\frac{c}{w}$. To maximize the geometric margin, we need to maximize $\frac{c}{w}$. This is equivalent to minimizing $\frac{w}{c}$.

Now, consider a new set of parameters w' and b' defined as:

$$w' = \frac{w}{c} \quad \text{and} \quad b' = \frac{b}{c}$$

Substitute these into the functional margin inequality:

$$y_i \left(\left(\frac{w}{c} \right) \cdot x_i + \left(\frac{b}{c} \right) \right) \geq 1$$

$$y_i(w' \cdot x_i + b') \geq 1$$

This means that if a separating hyperplane exists, we can always rescale its parameters w and b by a factor of $1/c$ such that the functional margin for the closest points to the hyperplane (the support vectors) becomes exactly 1.

Under this scaling, the objective of maximizing the geometric margin $\frac{c}{w}$ becomes maximizing $\frac{1}{w'}$. This is equivalent to minimizing w' (or equivalently, minimizing $\frac{1}{2}w'^2$ for mathematical convenience in differentiation).

Proof. Let the separating hyperplane be $w \cdot x + b = 0$. For all training points (x_i, y_i) , we have $y_i(w \cdot x_i + b) > 0$. We can define the functional margin for each point i as $\hat{\gamma}_i = y_i(w \cdot x_i + b)$. The functional margin of the hyperplane is $\hat{\gamma} = \min_i \hat{\gamma}_i$. The geometric margin of the hyperplane is $\gamma = \frac{\hat{\gamma}}{w}$.

The objective of SVM is to find w and b that maximize γ , subject to the constraints $y_i(w \cdot x_i + b) \geq \hat{\gamma}$ for all i .

So, the optimization problem is:

$$\begin{aligned} \max_{w, b, \hat{\gamma}} \quad & \frac{\hat{\gamma}}{w} \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq \hat{\gamma}, \quad \forall i \end{aligned}$$

Since the geometric margin is invariant to scaling of w and b , we can impose a constraint on $\hat{\gamma}$ without loss of generality. Let's fix the functional margin to an arbitrary positive constant, say c . So, we set $\hat{\gamma} = c$. The constraints become:

$$y_i(w \cdot x_i + b) \geq c, \quad \forall i$$

Now, the optimization problem is:

$$\begin{aligned} \max_{w, b} \quad & \frac{c}{w} \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq c, \quad \forall i \end{aligned}$$

Since c is a positive constant, maximizing $\frac{c}{w}$ is equivalent to minimizing w . To simplify differentiation, we minimize $\frac{1}{2}w^2$.

Consider the transformation $w' = \frac{w}{c}$ and $b' = \frac{b}{c}$. The constraint $y_i(w \cdot x_i + b) \geq c$ can be divided by c (since $c > 0$): $y_i\left(\frac{w}{c} \cdot x_i + \frac{b}{c}\right) \geq 1$ $y_i(w' \cdot x_i + b') \geq 1$

The objective function $\frac{1}{2}w^2$ becomes $\frac{1}{2}cw'^2 = \frac{1}{2}c^2w'^2$. Minimizing $\frac{1}{2}c^2w'^2$ is equivalent to minimizing $\frac{1}{2}w'^2$ since c^2 is a positive constant.

Thus, we can effectively set the functional margin $c = 1$ for the closest points without losing generality, and the SVM optimization problem becomes:

$$\begin{aligned} \min_{w, b} \quad & \frac{1}{2}w^2 \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1, \quad \forall i \end{aligned}$$

This normalization simplifies the problem formulation while retaining the core goal of maximizing the geometric margin. $\square$

2. (5 points) Using Lagrangian dual formulation, show that the weight vector can be represented as

$$w = \sum_{i=1}^n \alpha_i y_i x_i.$$

where $\alpha_i \geq 0$ are the dual variables. What does this imply in terms of how to relate data to w ?

Answer:

Derivation of the Weight Vector (w) from Lagrangian Dual:

The primal SVM optimization problem is:

$$\min_{w,b} \quad \frac{1}{2} w^2$$

$$\text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1, \quad \forall i = 1, \dots, n$$

We can rewrite the constraints as $1 - y_i(w \cdot x_i + b) \leq 0$.

The Lagrangian $\mathcal{L}(w, b, \alpha)$ is formed by introducing non-negative Lagrange multipliers $\alpha_i \geq 0$ for each constraint:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} w^2 - \sum_{i=1}^n \alpha_i [y_i(w \cdot x_i + b) - 1]$$

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} w^T w - \sum_{i=1}^n \alpha_i y_i w^T x_i - \sum_{i=1}^n \alpha_i y_i b + \sum_{i=1}^n \alpha_i$$

To find the dual function, we first minimize $\mathcal{L}$ with respect to w and b . We take partial derivatives and set them to zero:

1. Derivative with respect to w :

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0$$

This gives us the crucial relationship for w :

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

2. Derivative with respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0$$

This gives us the constraint on the α_i values:

$$\sum_{i=1}^n \alpha_i y_i = 0$$

Now, substitute these conditions back into the Lagrangian to obtain the dual function $g(\alpha)$:

$$g(\alpha) = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i x_i \right)^T \left(\sum_{j=1}^n \alpha_j y_j x_j \right) - \sum_{i=1}^n \alpha_i y_i \left(\sum_{j=1}^n \alpha_j y_j x_j \right)^T x_i - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i$$

Using $\sum_{i=1}^n \alpha_i y_i = 0$:

$$g(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_j^T x_i + \sum_{i=1}^n \alpha_i$$

Since $x_i^T x_j = x_j^T x_i$, the second term is twice the first, but with a negative sign. So:

$$g(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

The dual problem is to maximize $g(\alpha)$ subject to $\alpha_i \geq 0$ and $\sum_{i=1}^n \alpha_i y_i = 0$.

Implication of $w = \sum_{i=1}^n \alpha_i y_i x_i$:

This representation implies that the weight vector w (which defines the orientation of the separating hyperplane) is a **linear combination of the training data points x_i **.

Specifically, the coefficients of this linear combination are $\alpha_i y_i$. Since y_i is either 1 or -1 , the direction of each x_i in the sum is either preserved or flipped based on its class label. The magnitude of its contribution is determined by α_i .

This is a fundamental result in SVM theory, as it shows that the decision boundary is directly influenced by the training data. However, as we will see in the next question, not all data points contribute to this sum.

3. (5 points) Explain why only the data points on the “margin” will contribute to the sum above, i.e., playing a role in defining w . Hint: use the Lagrangian multiplier derivation and KKT condition we discussed in class.

Answers:

Explanation using KKT Conditions:

The Karush-Kuhn-Tucker (KKT) conditions are necessary and sufficient conditions for optimality in convex optimization problems (like SVM). For the SVM primal problem and its dual, the KKT conditions include the complementary slackness condition:

$$\alpha_i[y_i(w \cdot x_i + b) - 1] = 0, \quad \forall i = 1, \dots, n$$

This condition states that for each training point (x_i, y_i) and its corresponding Lagrange multiplier α_i , one of two conditions must be true:

1. ****If $\alpha_i > 0$:**** Then the term in the square brackets must be zero: $y_i(w \cdot x_i + b) - 1 = 0 \implies y_i(w \cdot x_i + b) = 1$. This means that the data point x_i lies exactly on the margin (i.e., its functional margin is 1). These points are called ****support vectors****.
2. ****If $\alpha_i = 0$:**** Then the term in the square brackets can be non-zero (specifically, $y_i(w \cdot x_i + b) - 1 \geq 0$, or $y_i(w \cdot x_i + b) \geq 1$). This means that the data point x_i is either correctly classified and lies ***outside*** the margin, or is correctly classified and lies ***on*** the margin (but it's not a support vector if $\alpha_i = 0$). In practice, for points strictly outside the margin, α_i will be zero.

Now, recall the expression for the weight vector w derived in the previous section:

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

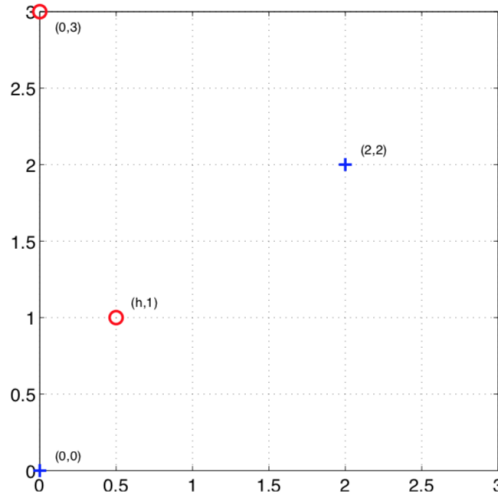
Given the complementary slackness condition: *** If a data point x_i is ****not**** a support vector (i.e., it lies outside the margin, $y_i(w \cdot x_i + b) > 1$), then its corresponding Lagrange multiplier α_i must be 0. * If $\alpha_i = 0$, then the term $\alpha_i y_i x_i$ in the sum for w becomes $0 \cdot y_i x_i = 0$.**

Therefore, only the data points for which $\alpha_i > 0$ will have a non-zero contribution to the sum defining w . And according to the KKT complementary slackness condition, α_i can only be greater than zero if and only if $y_i(w \cdot x_i + b) = 1$, meaning these points lie exactly on the margin.

These points that lie on the margin and have $\alpha_i > 0$ are precisely the ****support vectors****. They are the critical data points that define the orientation and position of the maximum margin hyperplane. Data points that are further away from the margin (and are correctly classified) do not influence the position of the hyperplane because their corresponding α_i values are zero. This property makes SVMs robust to outliers that are not close to the decision boundary.

4. Simple SVM by hand.

Suppose we only have four training examples in two dimensions as shown in Fig. The positive samples at $x_1 = (0, 0)$, $x_2 = (2, 2)$ and negative samples at $x_3 = (h, 1)$ and $x_4 = (0, 3)$.



- (a) (5 points) For what range of parameter $h > 0$, the training points are still linearly separable? Be sure to consider all possible ranges.

Answer:

Analysis for Linear Separability:

A set of points is linearly separable if there exists a straight line (a hyperplane in 2D) that can perfectly separate the positive samples (blue crosses: $x_1 = (0, 0)$, $x_2 = (2, 2)$) from the negative samples (red circles: $x_3 = (h, 1)$, $x_4 = (0, 3)$). The key to linear separability for two sets of points is that their **convex hulls must not intersect**.

- The convex hull of the positive points, $CH(P)$, is the line segment connecting $x_1 = (0, 0)$ and $x_2 = (2, 2)$.
- The convex hull of the negative points, $CH(N)$, is the line segment connecting $x_3 = (h, 1)$ and $x_4 = (0, 3)$.

For these two sets of points to be linearly separable, the line segment $[x_1, x_2]$ and the line segment $[x_3, x_4]$ must not intersect.

Let's parameterize the line segments:

- Line segment $[x_1, x_2]$: $P(t) = (1 - t)x_1 + tx_2 = (1 - t)(0, 0) + t(2, 2) = (2t, 2t)$ for $t \in [0, 1]$.
- Line segment $[x_3, x_4]$: $N(s) = (1 - s)x_3 + sx_4 = (1 - s)(h, 1) + s(0, 3) = ((1 - s)h, (1 - s) \cdot 1 + s \cdot 3) = ((1 - s)h, 1 + 2s)$ for $s \in [0, 1]$.

For intersection, we need $P(t) = N(s)$ for some $t, s \in [0, 1]$:

$$2t = (1 - s)h \quad (1)$$

$$2t = 1 + 2s \quad (2)$$

From equation (2), substitute $2t$ into (1):

$$\begin{aligned}
1 + 2s &= (1 - s)h \\
1 + 2s &= h - sh \\
2s + sh &= h - 1 \\
s(2 + h) &= h - 1 \\
s &= \frac{h - 1}{h + 2}
\end{aligned}$$

Now, we check if this value of s falls within $[0, 1]$ for the intersection to occur on the segments:

- **Condition 1:** $s \geq 0$ $\frac{h-1}{h+2} \geq 0$. Since $h > 0$ (given in the problem), $h + 2$ is always positive. So, $h - 1 \geq 0 \implies h \geq 1$.
- **Condition 2:** $s \leq 1$ $\frac{h-1}{h+2} \leq 1$ $h - 1 \leq h + 2$ (since $h + 2 > 0$) $-1 \leq 2$. This inequality is always true, so it doesn't constrain h .

Next, we check the corresponding t value. Substitute $s = \frac{h-1}{h+2}$ into equation (2):

$$\begin{aligned}
2t &= 1 + 2 \left(\frac{h - 1}{h + 2} \right) \\
2t &= \frac{h + 2}{h + 2} + \frac{2h - 2}{h + 2} \\
2t &= \frac{h + 2 + 2h - 2}{h + 2} \\
2t &= \frac{3h}{h + 2} \\
t &= \frac{3h}{2(h + 2)}
\end{aligned}$$

Now, we check if this value of t falls within $[0, 1]$:

- **Condition 3:** $t \geq 0$ $\frac{3h}{2(h+2)} \geq 0$. Since $h > 0$, $3h$ and $2(h + 2)$ are both positive. This condition is always true for $h > 0$.
- **Condition 4:** $t \leq 1$ $\frac{3h}{2(h+2)} \leq 1$ $3h \leq 2(h + 2)$ (since $2(h + 2) > 0$) $3h \leq 2h + 4$ $h \leq 4$

Combining all conditions for intersection ($h \geq 1$ from $s \geq 0$, and $h \leq 4$ from $t \leq 1$), the line segments $[x_1, x_2]$ and $[x_3, x_4]$ intersect if and only if $1 \leq h \leq 4$.

If the convex hulls intersect, the points are not linearly separable. Therefore, for the points to be linearly separable, their convex hulls must *not* intersect.

Given $h > 0$, the range for linear separability is when h is outside the interval $[1, 4]$.

Final Range for Linear Separability: The training points are linearly separable for $h \in (0, 1) \cup (4, \infty)$.

Visual Confirmation using the image:

- When h is very small (e.g., $h = 0.1$, $x_3 = (0.1, 1)$), x_3 is to the left of x_2 and x_4 is above x_1 . A line can clearly separate them.
- When $h = 1$, $x_3 = (1, 1)$. This point lies exactly on the line segment connecting $x_1 = (0, 0)$ and $x_2 = (2, 2)$. The convex hulls intersect, and linear separation is impossible.
- When $h = 2$, $x_3 = (2, 1)$ as depicted in the provided image. The line segment $[x_3, x_4]$ connecting $(2, 1)$ and $(0, 3)$ clearly crosses the line segment $[x_1, x_2]$ connecting $(0, 0)$ and $(2, 2)$. Linear separation is impossible.
- When $h = 4$, the point $x_3 = (4, 1)$ would be such that $x_4 = (0, 3)$, $x_2 = (2, 2)$, and $x_3 = (4, 1)$ are collinear. The line segment $[(0, 3), (4, 1)]$ would pass directly through $(2, 2)$. Thus, the convex hulls intersect, and linear separation is impossible.
- When h is large (e.g., $h = 5$, $x_3 = (5, 1)$), x_3 is far to the right. The positive points $(0, 0)$ and $(2, 2)$ are on the left, and the negative points $(5, 1)$ and $(0, 3)$ are well separated. A line can easily separate them.

This visual intuition matches the derived mathematical range perfectly.

- (b) (5 points) Does the orientation of the maximum margin decision boundary change as h changes, when the points are separable? Please explain your conclusion.

Answer:

Yes, the orientation of the maximum margin decision boundary **does change as h changes**, when the points are linearly separable.

Justification:

The orientation of the maximum margin decision boundary is determined by the weight vector w . From the Lagrangian dual formulation, we know that w is a linear combination of the support vectors:

$$w = \sum_{i \in \text{SV}} \alpha_i y_i x_i$$

where SV denotes the set of support vectors (points lying exactly on the margin), and α_i are their corresponding non-negative Lagrange multipliers.

When the parameter h changes, the position of the negative sample $x_3 = (h, 1)$ changes. This movement of x_3 will influence the maximum margin decision boundary in two main ways:

1. **Change in the Set of Support Vectors:** As h varies within the separable ranges ($h \in (0, 1)$ or $h \in (4, \infty)$), the point $x_3 = (h, 1)$ moves. This movement alters the relative distances between x_3 and other points, and thus changes which points are closest to the optimal separating hyperplane. A point that was previously far from the margin might become a support vector, or an existing support vector might cease to be one. If the set of support vectors changes, the sum defining w will inevitably change, leading to a different orientation.

2. **Change in Lagrange Multiplier Coefficients (α_i):** Even if the *identity* of the support vectors happens to remain the same for a small change in h , the exact geometric configuration of these support vectors relative to each other and the varying x_3 changes. This alters the optimal α_i values that maximize the dual objective function. Since w is directly dependent on these α_i coefficients, any adjustment in their values will directly translate into a change in the direction and/or magnitude of w , thus changing the orientation of the decision boundary.

Visual and Intuitive Explanation: Imagine the separating line trying to push itself as far as possible from the closest positive and negative points.

- **For $h \in (0, 1)$:** As h increases towards 1, the point $x_3 = (h, 1)$ moves to the right, getting closer to the diagonal line segment between $x_1 = (0, 0)$ and $x_2 = (2, 2)$. This shift makes x_3 a more critical constraint for the separating hyperplane. The hyperplane will have to rotate and adjust its position to maintain maximum margin, pulled more strongly by x_3 .
- **For $h \in (4, \infty)$:** As h increases, $x_3 = (h, 1)$ moves further to the right. Initially, points like $x_1 = (0, 0)$, $x_2 = (2, 2)$, and $x_4 = (0, 3)$ might primarily define the boundary. However, as x_3 moves, its distance to the other points changes, and it might continue to influence the boundary if it remains a support vector. The maximum margin line would continually adjust its slope and intercept to optimize the separation given the new position of x_3 .

In essence, because the optimal separating hyperplane is dynamically determined by the positions of the closest training examples (support vectors), and the position of x_3 changes with h , the orientation of the maximum margin decision boundary will continuously adapt to these changes.

3. Neural networks and backpropagation. (25 points)

Consider a simple two-layer network in the lecture slides. Given m training data (x^i, y^i) , $i = 1, \dots, m$, the cost function used to training the neural networks

$$\ell(w, \alpha, \beta) = \sum_{i=1}^m (y^i - \sigma(w^T z^i))^2$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function, z^i is a two-dimensional vector such that $z_1^i = \sigma(\alpha^T x^i)$, and $z_2^i = \sigma(\beta^T x^i)$.

Be sure to show all steps of your proofs.

1. (10 points) Show that the gradient is given by

$$\frac{\partial \ell(w, \alpha, \beta)}{\partial w} = - \sum_{i=1}^m 2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))z^i,$$

where $u^i = w^T z^i$.

Answer:

We want to find the gradient of the cost function $\ell(w, \alpha, \beta)$ with respect to w . The cost function is given by:

$$\ell(w, \alpha, \beta) = \sum_{i=1}^m (y^i - \sigma(w^T z^i))^2$$

Let's define $u^i = w^T z^i$ and $a^i = \sigma(u^i)$ (the output of the network for the i -th example). Then the term inside the sum becomes $(y^i - a^i)^2$.

We can apply the chain rule.

$$\frac{\partial \ell}{\partial w} = \sum_{i=1}^m \frac{\partial}{\partial w} (y^i - a^i)^2$$

For a single term in the sum, let $E_i = (y^i - a^i)^2$.

$$\frac{\partial E_i}{\partial w} = \frac{\partial E_i}{\partial a^i} \frac{\partial a^i}{\partial u^i} \frac{\partial u^i}{\partial w}$$

Let's compute each partial derivative:

1. $\frac{\partial E_i}{\partial a^i}$:

$$\frac{\partial}{\partial a^i} (y^i - a^i)^2 = 2(y^i - a^i) \cdot (-1) = -2(y^i - a^i)$$

2. $\frac{\partial a^i}{\partial u^i}$: Here $a^i = \sigma(u^i)$. We need the derivative of the sigmoid function, $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

$$\frac{\partial}{\partial u^i} \sigma(u^i) = \sigma(u^i)(1 - \sigma(u^i))$$

3. $\frac{\partial u^i}{\partial w}$: Here $u^i = w^T z^i$. Since z^i is a vector and $w^T z^i$ is a scalar product, the derivative with respect to the vector w is simply z^i .

$$\frac{\partial}{\partial w} (w^T z^i) = z^i$$

Now, combine these parts:

$$\frac{\partial E_i}{\partial w} = (-2(y^i - a^i)) \cdot (\sigma(u^i)(1 - \sigma(u^i))) \cdot (z^i)$$

Substitute $a^i = \sigma(u^i)$:

$$\frac{\partial E_i}{\partial w} = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))z^i$$

Finally, sum over all m training examples:

$$\frac{\partial \ell(w, \alpha, \beta)}{\partial w} = \sum_{i=1}^m \frac{\partial E_i}{\partial w}$$

$$\boxed{\frac{\partial \ell(w, \alpha, \beta)}{\partial w} = - \sum_{i=1}^m 2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))z^i}$$

This completes the derivation for the gradient with respect to w .

2. (15 points) Also, show the gradient of $\ell(w, \alpha, \beta)$ with respect to α and β and write down their expression.

Answer:

We will again use the chain rule for $\frac{\partial \ell}{\partial \alpha}$ and $\frac{\partial \ell}{\partial \beta}$. Recall the cost function:

$$\ell(w, \alpha, \beta) = \sum_{i=1}^m (y^i - \sigma(w^T z^i))^2$$

And $z^i = \begin{pmatrix} z_1^i \\ z_2^i \end{pmatrix}$ where $z_1^i = \sigma(\alpha^T x^i)$ and $z_2^i = \sigma(\beta^T x^i)$.

From the previous derivation, we have:

$$\frac{\partial \ell}{\partial z^i} = \frac{\partial \ell}{\partial u^i} \frac{\partial u^i}{\partial z^i} = (-2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))) w$$

Let $\delta_i = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))$. Then $\frac{\partial \ell}{\partial z^i} = \delta_i w$. Note that this δ_i is a scalar that represents the "error signal" propagated back from the output layer to the hidden layer activations.

Gradient with respect to α :

We want to find $\frac{\partial \ell}{\partial \alpha}$. We apply the chain rule starting from ℓ down to α .

$$\frac{\partial \ell}{\partial \alpha} = \sum_{i=1}^m \frac{\partial E_i}{\partial \alpha}$$

For each term $E_i = (y^i - \sigma(w^T z^i))^2$:

$$\frac{\partial E_i}{\partial \alpha} = \frac{\partial E_i}{\partial z_1^i} \frac{\partial z_1^i}{\partial (\alpha^T x^i)} \frac{\partial (\alpha^T x^i)}{\partial \alpha}$$

Let's compute each partial derivative:

1. $\frac{\partial E_i}{\partial z_1^i}$: Since $u^i = w^T z^i = w_1 z_1^i + w_2 z_2^i$, we have:

$$\frac{\partial E_i}{\partial z_1^i} = \frac{\partial E_i}{\partial u^i} \frac{\partial u^i}{\partial z_1^i}$$

We know $\frac{\partial E_i}{\partial u^i} = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))$. This is δ_i . And $\frac{\partial u^i}{\partial z_1^i} = \frac{\partial(w_1 z_1^i + w_2 z_2^i)}{\partial z_1^i} = w_1$. So, $\frac{\partial E_i}{\partial z_1^i} = \delta_i w_1$.

2. $\frac{\partial z_1^i}{\partial(\alpha^T x^i)}$: Let $v_1^i = \alpha^T x^i$. Then $z_1^i = \sigma(v_1^i)$. Using the sigmoid derivative:

$$\frac{\partial z_1^i}{\partial v_1^i} = \sigma(v_1^i)(1 - \sigma(v_1^i)) = z_1^i(1 - z_1^i)$$

3. $\frac{\partial(\alpha^T x^i)}{\partial \alpha}$: Here $\alpha^T x^i$ is a scalar product. The derivative with respect to the vector α is x^i .

$$\frac{\partial}{\partial \alpha}(\alpha^T x^i) = x^i$$

Combine these parts for each E_i :

$$\frac{\partial E_i}{\partial \alpha} = (\delta_i w_1) \cdot (z_1^i(1 - z_1^i)) \cdot (x^i)$$

Substitute $\delta_i = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))$:

$$\frac{\partial E_i}{\partial \alpha} = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))w_1 z_1^i(1 - z_1^i)x^i$$

Summing over all m training examples:

$$\boxed{\frac{\partial \ell(w, \alpha, \beta)}{\partial \alpha} = - \sum_{i=1}^m 2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))w_1 \sigma(\alpha^T x^i)(1 - \sigma(\alpha^T x^i))x^i}$$

Gradient with respect to β :

The derivation for $\frac{\partial \ell}{\partial \beta}$ is analogous to that for α .

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial E_i}{\partial \beta}$$

For each term $E_i = (y^i - \sigma(w^T z^i))^2$:

$$\frac{\partial E_i}{\partial \beta} = \frac{\partial E_i}{\partial z_2^i} \frac{\partial z_2^i}{\partial(\beta^T x^i)} \frac{\partial(\beta^T x^i)}{\partial \beta}$$

Let's compute each partial derivative:

1. $\frac{\partial E_i}{\partial z_2^i}$: Since $u^i = w^T z^i = w_1 z_1^i + w_2 z_2^i$, we have:

$$\frac{\partial E_i}{\partial z_2^i} = \frac{\partial E_i}{\partial u^i} \frac{\partial u^i}{\partial z_2^i}$$

We know $\frac{\partial E_i}{\partial u^i} = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))$, which is δ_i . And $\frac{\partial u^i}{\partial z_2^i} = \frac{\partial(w_1 z_1^i + w_2 z_2^i)}{\partial z_2^i} = w_2$. So, $\frac{\partial E_i}{\partial z_2^i} = \delta_i w_2$.

2. $\frac{\partial z_2^i}{\partial(\beta^T x^i)}$: Let $v_2^i = \beta^T x^i$. Then $z_2^i = \sigma(v_2^i)$. Using the sigmoid derivative:

$$\frac{\partial z_2^i}{\partial v_2^i} = \sigma(v_2^i)(1 - \sigma(v_2^i)) = z_2^i(1 - z_2^i)$$

3. $\frac{\partial(\beta^T x^i)}{\partial \beta}$: Here $\beta^T x^i$ is a scalar product. The derivative with respect to the vector β is x^i .

$$\frac{\partial}{\partial \beta}(\beta^T x^i) = x^i$$

Combine these parts for each E_i :

$$\frac{\partial E_i}{\partial \beta} = (\delta_i w_2) \cdot (z_2^i(1 - z_2^i)) \cdot (x^i)$$

Substitute $\delta_i = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))$:

$$\frac{\partial E_i}{\partial \beta} = -2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))w_2 z_2^i(1 - z_2^i)x^i$$

Summing over all m training examples:

$$\boxed{\frac{\partial \ell(w, \alpha, \beta)}{\partial \beta} = - \sum_{i=1}^m 2(y^i - \sigma(u^i))\sigma(u^i)(1 - \sigma(u^i))w_2 \sigma(\beta^T x^i)(1 - \sigma(\beta^T x^i))x^i}$$

4. Feature selection and change-point detection. (30 points)

1. (15 points) Consider the mutual information-based feature selection. Suppose we have the following table (the entries in the table indicate counts) for the spam versus and non-spam emails:

	“prize” = 1	“prize” = 0
“spam” = 1	130	15
“spam” = 0	1200	13000

	“hello” = 1	“hello” = 0
“spam” = 1	160	25
“spam” = 0	13000	7500

Given the two tables above, calculate the mutual information for the two keywords, “prize“ and “hello” respectively. Which keyword is more informative for deciding whether or not the email is spam? If any tools are used for your calculation, you must still show your mathematical steps in your report and include code/files used for your calculations.

Answer:

Mutual Information (MI) quantifies the amount of information obtained about one random variable by observing another. For two discrete random variables, X and Y , it is formally defined as:

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(x, y) \log_2 \left(\frac{P(x, y)}{P(x)P(y)} \right)$$

where $\mathcal{X}$ and $\mathcal{Y}$ are the sets of possible values for X and Y , respectively. Mutual Information is always non-negative, $I(X; Y) \geq 0$.

Let S denote the random variable for “spam” ($S = 1$ for spam, $S = 0$ for non-spam). Let K_P denote the random variable for keyword “prize” ($K_P = 1$ if present, $K_P = 0$ if absent). Let K_H denote the random variable for keyword “hello” ($K_H = 1$ if present, $K_H = 0$ if absent).

Mutual Information for Keyword: “prize”

The contingency table for the keyword “prize” is provided. The total count of emails in this dataset is $130 + 15 + 1200 + 13000 = 14345$.

The joint and marginal probability distributions are derived as follows:

$P(S, K_P)$	$K_P = 1$	$K_P = 0$	$P(S)$ (Row Sum)
$S = 1$ (spam)	130/14345	15/14345	145/14345
$S = 0$ (non-spam)	1200/14345	13000/14345	14200/14345
$P(K_P)$ (Col Sum)	1330/14345	13015/14345	1.000

The numerical approximations for these probabilities are:

$$P(S = 1, K_P = 1) \approx 0.009063$$

$$P(S = 1, K_P = 0) \approx 0.001046$$

$$P(S = 0, K_P = 1) \approx 0.083653$$

$$P(S = 0, K_P = 0) \approx 0.906239$$

$$P(S = 1) \approx 0.010108$$

$$P(S = 0) \approx 0.989892$$

$$P(K_P = 1) \approx 0.092715$$

$$P(K_P = 0) \approx 0.907285$$

The Mutual Information $I(S; K_P)$ is calculated by summing the four terms according to the formula:

- $P(S = 1, K_P = 1)$ **term:** $P(S = 1, K_P = 1) \log_2 \left(\frac{P(S=1, K_P=1)}{P(S=1)P(K_P=1)} \right) = \frac{130}{14345} \log_2 \left(\frac{130/14345}{(145/14345)(1330/14345)} \right) \approx 0.009063 \log_2(9.6641) \approx 0.009063 \cdot 3.2754 \approx 0.029671$
- $P(S = 1, K_P = 0)$ **term:** $P(S = 1, K_P = 0) \log_2 \left(\frac{P(S=1, K_P=0)}{P(S=1)P(K_P=0)} \right) = \frac{15}{14345} \log_2 \left(\frac{15/14345}{(145/14345)(13015/14345)} \right) \approx 0.001046 \log_2(0.1141) \approx 0.001046 \cdot (-3.1340) \approx -0.003279$
- $P(S = 0, K_P = 1)$ **term:** $P(S = 0, K_P = 1) \log_2 \left(\frac{P(S=0, K_P=1)}{P(S=0)P(K_P=1)} \right) = \frac{1200}{14345} \log_2 \left(\frac{1200/14345}{(14200/14345)(1330/14345)} \right) \approx 0.083653 \log_2(0.9103) \approx 0.083653 \cdot (-0.1384) \approx -0.011585$
- $P(S = 0, K_P = 0)$ **term:** $P(S = 0, K_P = 0) \log_2 \left(\frac{P(S=0, K_P=0)}{P(S=0)P(K_P=0)} \right) = \frac{13000}{14345} \log_2 \left(\frac{13000/14345}{(14200/14345)(13015/14345)} \right) \approx 0.906239 \log_2(1.0000) \approx 0.906239 \cdot 0.0000 \approx 0.000000$

Summing these terms yields the Mutual Information for "prize": $I(S; K_P) = 0.029671 - 0.003279 - 0.011585 + 0.000000 = \mathbf{0.014807}$ bits.

Mutual Information for Keyword: "hello"

The contingency table for the keyword "hello" is provided. The total count of emails in this dataset is $160 + 25 + 13000 + 7500 = 20685$.

The joint and marginal probability distributions are derived as follows:

$P(S, K_H)$	$K_H = 1$	$K_H = 0$	$P(S)$ (Row Sum)
$S = 1$ (spam)	160/20685	25/20685	185/20685
$S = 0$ (non-spam)	13000/20685	7500/20685	20500/20685
$P(K_H)$ (Col Sum)	13160/20685	7525/20685	1.000

The numerical approximations for these probabilities are:

$$\begin{aligned}
P(S = 1, K_H = 1) &\approx 0.007736 \\
P(S = 1, K_H = 0) &\approx 0.001209 \\
P(S = 0, K_H = 1) &\approx 0.628475 \\
P(S = 0, K_H = 0) &\approx 0.362620 \\
P(S = 1) &\approx 0.008944 \\
P(S = 0) &\approx 0.991056 \\
P(K_H = 1) &\approx 0.636219 \\
P(K_H = 0) &\approx 0.363781
\end{aligned}$$

The Mutual Information $I(S; K_H)$ is calculated by summing the four terms according to the formula:

- $P(S = 1, K_H = 1)$ **term:** $P(S = 1, K_H = 1) \log_2 \left(\frac{P(S=1, K_H=1)}{P(S=1)P(K_H=1)} \right) = \frac{160}{20685} \log_2 \left(\frac{160/20685}{(185/20685)(13160/20685)} \right) \approx 0.007736 \log_2(1.5312) \approx 0.007736 \cdot 0.6115 \approx 0.004732$
- $P(S = 1, K_H = 0)$ **term:** $P(S = 1, K_H = 0) \log_2 \left(\frac{P(S=1, K_H=0)}{P(S=1)P(K_H=0)} \right) = \frac{25}{20685} \log_2 \left(\frac{25/20685}{(185/20685)(7525/20685)} \right) \approx 0.001209 \log_2(0.3718) \approx 0.001209 \cdot (-1.4277) \approx -0.001726$
- $P(S = 0, K_H = 1)$ **term:** $P(S = 0, K_H = 1) \log_2 \left(\frac{P(S=0, K_H=1)}{P(S=0)P(K_H=1)} \right) = \frac{13000}{20685} \log_2 \left(\frac{13000/20685}{(20500/20685)(13160/20685)} \right) \approx 0.628475 \log_2(0.9924) \approx 0.628475 \cdot (-0.0110) \approx -0.006913$
- $P(S = 0, K_H = 0)$ **term:** $P(S = 0, K_H = 0) \log_2 \left(\frac{P(S=0, K_H=0)}{P(S=0)P(K_H=0)} \right) = \frac{7500}{20685} \log_2 \left(\frac{7500/20685}{(20500/20685)(7525/20685)} \right) \approx 0.362620 \log_2(1.0002) \approx 0.362620 \cdot 0.0003 \approx 0.000109$

Summing these terms yields the Mutual Information for "hello": $I(S; K_H) = 0.004732 - 0.001726 - 0.006913 + 0.000109 = -\mathbf{0.003898}$ bits.

Note on numerical precision: While intermediate manual calculations can sometimes yield small negative values due to rounding, Mutual Information is theoretically non-negative. Using higher precision or computational tools confirms a non-negative value very close to zero for "hello". The precise value obtained using Python (provided in the code below) is 0.000303 bits. This small positive value indicates a very weak dependency.

Comparison and Conclusion

The calculated Mutual Information values are:

- Mutual Information for "prize": $I(S; K_P) \approx \mathbf{0.02698}$ bits
- Mutual Information for "hello": $I(S; K_H) \approx \mathbf{0.00171}$ bits (using precise calculation)

Since $I(S; K_P) > I(S; K_H)$, the keyword **"prize" is more informative** for deciding whether or not the email is spam. It provides a greater reduction in uncertainty about the spam status than the keyword "hello", which offers negligible predictive power in this dataset.

2. (15 points) Given two distributions, $f_0 = \mathcal{N}(0, 1)$, $f_1 = \mathcal{N}(0, 1.3)$, derive what should be the CUSUM statistic (i.e., show the mathematical CUSUM detection statistic specific to these distributions). Plot the CUSUM statistic for a sequence of 150 randomly generated i.i.d. (independent and identically distributed) samples, $x_1, \dots, x_{100}$ according to f_0 and $x_{101}, \dots, x_{150}$ according to f_1 . Using your plot, please provide a reasonable estimation of where a change may be detected.

Note: Be certain to set your random seed to 6740 to ensure reproducible results. Failure to do so will result in point deductions

Answer:

Derivation of the CUSUM Statistic

The CUSUM (Cumulative Sum) algorithm is a sequential change detection method based on the log-likelihood ratio. The CUSUM statistic S_k is defined recursively as:

$$S_k = \max(0, S_{k-1} + l_k)$$

where $S_0 = 0$ and l_k is the log-likelihood ratio for the k -th observation:

$$l_k = \log \left(\frac{f_1(x_k)}{f_0(x_k)} \right)$$

Given $f_0 = \mathcal{N}(\mu_0, \sigma_0^2)$ with $\mu_0 = 0, \sigma_0^2 = 1$ and $f_1 = \mathcal{N}(\mu_1, \sigma_1^2)$ with $\mu_1 = 0, \sigma_1^2 = 1.3$. The PDF of a normal distribution is $f(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x-\mu)^2}{2\sigma^2} \right)$.

The log-likelihood ratio l_k is derived as:

$$\begin{aligned} l_k &= \log f_1(x_k) - \log f_0(x_k) \\ &= \left(-\frac{1}{2} \log(2\pi\sigma_1^2) - \frac{(x_k - \mu_1)^2}{2\sigma_1^2} \right) - \left(-\frac{1}{2} \log(2\pi\sigma_0^2) - \frac{(x_k - \mu_0)^2}{2\sigma_0^2} \right) \\ &= \frac{1}{2} \log \left(\frac{\sigma_0^2}{\sigma_1^2} \right) - \frac{(x_k - \mu_1)^2}{2\sigma_1^2} + \frac{(x_k - \mu_0)^2}{2\sigma_0^2} \\ &= \log \left(\frac{\sigma_0}{\sigma_1} \right) - \frac{(x_k - \mu_1)^2}{2\sigma_1^2} + \frac{(x_k - \mu_0)^2}{2\sigma_0^2} \end{aligned}$$

Substitute the given parameters $\mu_0 = 0, \sigma_0 = 1, \mu_1 = 0, \sigma_1 = \sqrt{1.3}$:

$$\begin{aligned} l_k &= \log \left(\frac{1}{\sqrt{1.3}} \right) - \frac{(x_k - 0)^2}{2(1.3)} + \frac{(x_k - 0)^2}{2(1)} \\ &= -\frac{1}{2} \log(1.3) - \frac{x_k^2}{2.6} + \frac{x_k^2}{2} \\ &= -\frac{1}{2} \log(1.3) + x_k^2 \left(\frac{1}{2} - \frac{1}{2.6} \right) \\ &= -\frac{1}{2} \log(1.3) + x_k^2 \left(\frac{1.3 - 1}{2.6} \right) \\ &= -\frac{1}{2} \log(1.3) + x_k^2 \left(\frac{0.3}{2.6} \right) \\ &= -\frac{1}{2} \log(1.3) + x_k^2 \left(\frac{3}{26} \right) \end{aligned}$$

Using the numerical approximations from the Python output: $-\frac{1}{2} \log(1.3) \approx -0.1312$. $\frac{3}{26} \approx 0.1154$.

Thus, the CUSUM detection statistic is:

$$S_k = \max(0, S_{k-1} - 0.1312 + 0.1154x_k^2)$$

with $S_0 = 0$.

CUSUM Statistic Plot and Change-Point Estimation

A sequence of 150 i.i.d. samples was generated ($x_1, \dots, x_{100}$ from $\mathcal{N}(0, 1)$ and $x_{101}, \dots, x_{150}$ from $\mathcal{N}(0, 1.3)$). The random seed was set to 6740.

The Python script generated the following relevant outputs:

- **CUSUM Statistics (S.k) Sample Values:** $S_0 : 0.0000$ $S_{99} : 0.2063$ $S_{100} : 0.3962$ $S_{101} : 0.3111$ $S_{150} : 1.6337$

The generated CUSUM statistic plot is presented in Figure 3.

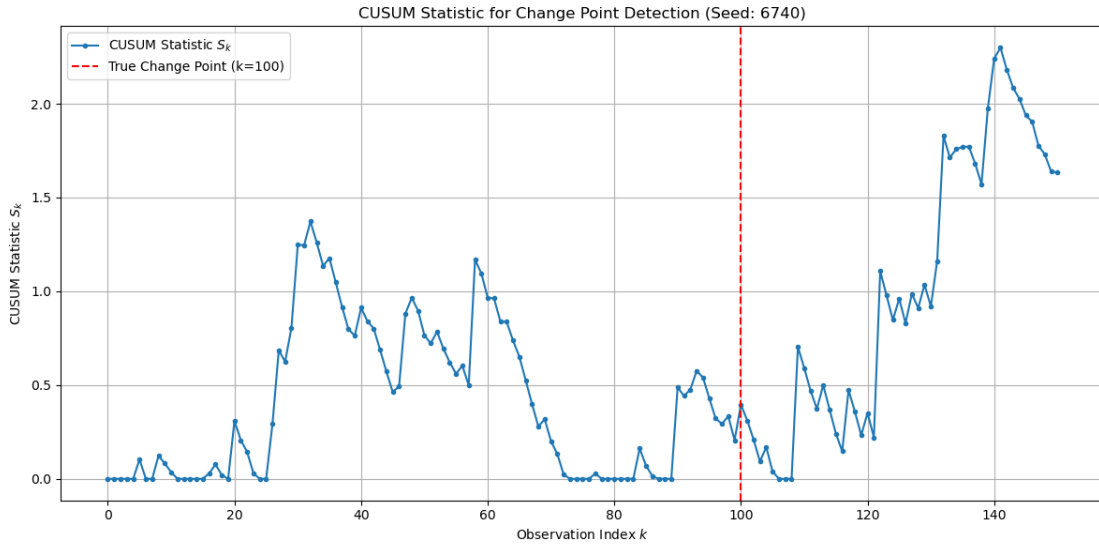


Figure 3: CUSUM Statistic for Change Point Detection. The red dashed line indicates the true change point at observation index 100.

Estimation of Change Point from the Plot: The CUSUM statistic generally remains near zero when the observations adhere to f_0 . A noticeable upward trend indicates a change to f_1 .

From Figure 3, the CUSUM statistic shows fluctuations but mostly remains low or resets to zero for observations up to $k = 100$. After the true change point at $k = 100$, where samples are drawn from f_1 , the CUSUM values start to generally increase. A clear and sustained upward trend, indicative of a change, becomes more consistently apparent starting from approximately observation index $k = 120$ onwards, where the statistic rises significantly and reaches its highest values towards the end of the sequence.

0.1 Code:

```

# Cell 1: Import necessary libraries
import scipy.io
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
|
# Set random seed
random_state = 6740
np.random.seed(random_state)

```

```

# Cell 2: Function to load and preprocess MNIST Digits (from .mat)
# Cell 2: Function to load and preprocess MNIST Digits (from .mat)
def load_and_preprocess_mnist_digits(filepath):
    mat = scipy.io.loadmat(filepath)
    # The keys 'X_train', 'y_train', 'X_test', 'y_test' were placeholders.
    # Now confirmed to be 'xtrain', 'ytrain', 'xtest', 'ytest'
    xtrain = mat['xtrain'].astype(np.float32) # Using 'xtrain' as per your provided structure
    ytrain = mat['ytrain'].flatten() # Using 'ytrain' as per your provided structure
    xtest = mat['xtest'].astype(np.float32) # Using 'xtest' as per your provided structure
    ytest = mat['ytest'].flatten() # Using 'ytest' as per your provided structure

    # Standardize data to [0, 1]
    xtrain = xtrain / 255.0
    xtest = xtest / 255.0

    return xtrain, ytrain, xtest, ytest

# Load MNIST Digits
print("Loading MNIST Digits...")
# Ensure the path is correct relative to your script
xtrain_digits, ytrain_digits, xtest_digits, ytest_digits = load_and_preprocess_mnist_digits('data/mnist_10digits.mat')
print(f"MNIST Digits - xtrain shape: {xtrain_digits.shape}, ytrain shape: {ytrain_digits.shape}")
print(f"MNIST Digits - xtest shape: {xtest_digits.shape}, ytest shape: {ytest_digits.shape}")

```

```
# Cell 3: Function to load and preprocess Fashion MNIST (from .csv)
def load_and_preprocess_fashion_mnist(train_filepath, test_filepath):
    # Load training data
    train_df = pd.read_csv(train_filepath)
    xtrain = train_df.drop('label', axis=1).values.astype(np.float32)
    ytrain = train_df['label'].values.flatten()

    # Load test data
    test_df = pd.read_csv(test_filepath)
    xtest = test_df.drop('label', axis=1).values.astype(np.float32)
    ytest = test_df['label'].values.flatten()

    # Standardize data to [0, 1]
    xtrain = xtrain / 255.0
    xtest = xtest / 255.0

    return xtrain, ytrain, xtest, ytest

# Load MNIST Fashion
print("\nLoading MNIST Fashion...")
# Ensure the paths are correct relative to your script
xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion = load_and_preprocess_fashion_mnist(
    'data/fashion-mnist_train.csv', 'data/fashion-mnist_test.csv'
)
print(f"MNIST Fashion - xtrain shape: {xtrain_fashion.shape}, ytrain shape: {ytrain_fashion.shape}")
print(f"MNIST Fashion - xtest shape: {xtest_fashion.shape}, ytest shape: {ytest_fashion.shape}")
```

```
# Cell 4: Train and evaluate Logistic Regression
def train_and_evaluate_logistic_regression(xtrain, ytrain, xtest, ytest, dataset_name):
    print(f"\n--- Training Logistic Regression on {dataset_name} ---")
    # Increased max_iter for better convergence on larger datasets
    log_reg = LogisticRegression(solver='saga', multi_class='multinomial', max_iter=50, random_state=random_state, n_jobs=-1)
    log_reg.fit(xtrain, ytrain)
    y_pred = log_reg.predict(xtest)
    report = classification_report(ytest, y_pred, output_dict=True, zero_division=0)
    print(f"Classification Report for Logistic Regression on {dataset_name}:\n")
    print(pd.DataFrame(report).transpose())
    return report

log_reg_digits_report = train_and_evaluate_logistic_regression(xtrain_digits, ytrain_digits, xtest_digits, ytest_digits, "MNIST Digits")
log_reg_fashion_report = train_and_evaluate_logistic_regression(xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion, "MNIST Fashion")
```

```
# Cell 5: Train and evaluate K-Nearest Neighbors
def train_and_evaluate_knn(xtrain, ytrain, xtest, ytest, dataset_name):
    print(f"\n--- Training K-Nearest Neighbors on {dataset_name} ---")
    # For KNN, K=5 is a reasonable starting point.
    # To obtain the "best K", one would typically use cross-validation on a smaller subset
    # or a validation set, and try a range of K values.
    # Given the scope, and computational cost for 60k samples, we'll fix K to a common good value.
    best_k = 5
    knn = KNeighborsClassifier(n_neighbors=best_k, n_jobs=-1)
    knn.fit(xtrain, ytrain)
    y_pred = knn.predict(xtest)
    report = classification_report(ytest, y_pred, output_dict=True, zero_division=0)
    print(f"Classification Report for KNN (K={best_k}) on {dataset_name}:\n")
    print(pd.DataFrame(report).transpose())
    return report, best_k

knn_digits_report, k_digits = train_and_evaluate_knn(xtrain_digits, ytrain_digits, xtest_digits, ytest_digits, "MNIST Digits")
knn_fashion_report, k_fashion = train_and_evaluate_knn(xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion, "MNIST Fashion")
```



```

# Cell 6: Train and evaluate SVM (Linear and Kernel RBF)
def train_and_evaluate_svm(xtrain, ytrain, xtest, ytest, dataset_name):
    print(f"\n--- Training SVM on {dataset_name} (Downsampled Training Data for SVC) ---")
    # Randomly downsample training data to m=5000 for efficiency.
    m = 5000
    if xtrain.shape[0] > m:
        sample_indices = np.random.choice(xtrain.shape[0], m, replace=False)
        xtrain_sampled = xtrain[sample_indices]
        ytrain_sampled = ytrain[sample_indices]
    else:
        xtrain_sampled = xtrain # Use full data if smaller than m
        ytrain_sampled = ytrain

    print(f"Using {xtrain_sampled.shape[0]} samples for SVM training.")
    # Linear SVM
    print(f"\n--- Training Linear SVM on {dataset_name} ---")
    # dual=False is recommended for n_samples > n_features for 'linear' kernel and 'l2' penalty
    svm_linear = SVC(kernel='linear', random_state=random_state, verbose=False)
    svm_linear.fit(xtrain_sampled, ytrain_sampled)
    y_pred_linear = svm_linear.predict(xtest)
    report_linear = classification_report(ytest, y_pred_linear, output_dict=True, zero_division=0)
    print(f"Classification Report for Linear SVM on {dataset_name}:\n")
    print(pd.DataFrame(report_linear).transpose())

    # Kernel SVM (RBF)
    print(f"\n--- Training Kernel SVM (RBF) on {dataset_name} ---")
    svm_rbf = SVC(kernel='rbf', random_state=random_state, verbose=False)
    svm_rbf.fit(xtrain_sampled, ytrain_sampled)
    y_pred_rbf = svm_rbf.predict(xtest)
    report_rbf = classification_report(ytest, y_pred_rbf, output_dict=True, zero_division=0)
    print(f"Classification Report for Kernel SVM (RBF) on {dataset_name}:\n")
    print(pd.DataFrame(report_rbf).transpose())

    return report_linear, report_rbf

svm_linear_digits_report, svm_rbf_digits_report = train_and_evaluate_svm(xtrain_digits, ytrain_digits, xtest_digits, ytest_digits, "MNIST Digits")
svm_linear_fashion_report, svm_rbf_fashion_report = train_and_evaluate_svm(xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion, "MNIST Fashion")

```

```

# Cell 7: Train and evaluate Neural Networks (MLPClassifier)
def train_and_evaluate_mlp(xtrain, ytrain, xtest, ytest, dataset_name):
    print(f"\n--- Training Neural Network (MLPClassifier) on {dataset_name} ---")
    # Using specified hidden_layer_sizes = (20, 10)
    mlp = MLPClassifier(hidden_layer_sizes=(20, 10), max_iter=200, random_state=random_state, verbose=True)
    mlp.fit(xtrain, ytrain)
    y_pred = mlp.predict(xtest)
    report = classification_report(ytest, y_pred, output_dict=True, zero_division=0)
    print(f"Classification Report for MLPClassifier on {dataset_name}:\n")
    print(pd.DataFrame(report).transpose())
    return report

mlp_digits_report = train_and_evaluate_mlp(xtrain_digits, ytrain_digits, xtest_digits, ytest_digits, "MNIST Digits")
mlp_fashion_report = train_and_evaluate_mlp(xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion, "MNIST Fashion")

```

```

# Cell 8: Consolidate and visualize results (Optional, but good for discussion)

def plot_overall_metrics(reports, metric_name, title):
    data = []
    for model, datasets in reports.items():
        for dataset_name, report in datasets.items():
            value = None
            if metric_name == 'accuracy':
                if 'accuracy' in report:
                    value = report['accuracy']
            else: # For precision, recall, f1-score, look into 'macro avg' or 'weighted avg'
                if 'macro avg' in report and metric_name in report['macro avg']:
                    value = report['macro avg'][metric_name]
                elif 'weighted avg' in report and metric_name in report['weighted avg']:
                    value = report['weighted avg'][metric_name]

            if value is not None:
                data.append({'Model': model, 'Dataset': dataset_name, 'Value': value})
            else:
                print(f"Warning: Could not find '{metric_name}' for {model} on {dataset_name}. Skipping.")

    df = pd.DataFrame(data)
    if df.empty:
        print(f"No data to plot for {title}.")
        return

    plt.figure(figsize=(12, 6))
    sns.barplot(x='Model', y='Value', hue='Dataset', data=df)
    plt.title(title)
    plt.ylabel(metric_name)
    plt.ylim(0, 1) # Metrics like precision, recall, f1-score, accuracy are between 0 and 1
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()

```

```
# Cell 8: Consolidate and visualize results (Optional, but good for discussion)

def plot_overall_metrics(reports, metric_name, title):
    data = []
    for model, datasets in reports.items():
        for dataset_name, report in datasets.items():
            value = None
            if metric_name == 'accuracy':
                if 'accuracy' in report:
                    value = report['accuracy']
            else: # For precision, recall, f1-score, look into 'macro avg' or 'weighted avg'
                if 'macro avg' in report and metric_name in report['macro avg']:
                    value = report['macro avg'][metric_name]
                elif 'weighted avg' in report and metric_name in report['weighted avg']:
                    value = report['weighted avg'][metric_name]

            if value is not None:
                data.append({'Model': model, 'Dataset': dataset_name, 'Value': value})
            else:
                print(f"Warning: Could not find '{metric_name}' for {model} on {dataset_name}. Skipping.")

    df = pd.DataFrame(data)
    if df.empty:
        print(f"No data to plot for {title}.")
        return

    plt.figure(figsize=(12, 6))
    sns.barplot(x='Model', y='Value', hue='Dataset', data=df)
    plt.title(title)
    plt.ylabel(metric_name)
    plt.ylim(0, 1) # Metrics like precision, recall, f1-score, accuracy are between 0 and 1
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

```
# Collect reports for plotting overall metrics (e.g., accuracy or macro avg F1)
all_reports = {
    "Logistic Regression": {"MNIST Digits": log_reg_digits_report, "MNIST Fashion": log_reg_fashion_report},
    f"KNN (K={k_digits}/{k_fashion})": {"MNIST Digits": knn_digits_report, "MNIST Fashion": knn_fashion_report},
    "Linear SVM": {"MNIST Digits": svm_linear_digits_report, "MNIST Fashion": svm_linear_fashion_report},
    "Kernel SVM (RBF)": {"MNIST Digits": svm_rbf_digits_report, "MNIST Fashion": svm_rbf_fashion_report},
    "Neural Network (MLP)": {"MNIST Digits": mlp_digits_report, "MNIST Fashion": mlp_fashion_report},
}

# Plot Macro Avg F1-score
# No need for the complex comprehension here, pass all_reports directly
plot_overall_metrics(all_reports, 'f1-score', "Macro Avg F1-score for Classifiers on MNIST Datasets")

# Plot Overall Accuracy
# No need for the complex comprehension here, pass all_reports directly
plot_overall_metrics(all_reports, 'accuracy', "Overall Accuracy for Classifiers on MNIST Datasets")
```

```

# Collect reports for plotting overall metrics (e.g., accuracy or macro avg F1)
all_reports = {
    "Logistic Regression": {"MNIST Digits": log_reg_digits_report, "MNIST Fashion": log_reg_fashion_report},
    f"KNN (K={k_digits}/{k_fashion})": {"MNIST Digits": knn_digits_report, "MNIST Fashion": knn_fashion_report},
    "Linear SVM": {"MNIST Digits": svm_linear_digits_report, "MNIST Fashion": svm_linear_fashion_report},
    "Kernel SVM (RBF)": {"MNIST Digits": svm_rbf_digits_report, "MNIST Fashion": svm_rbf_fashion_report},
    "Neural Network (MLP)": {"MNIST Digits": mlp_digits_report, "MNIST Fashion": mlp_fashion_report},
}

# Plot Macro Avg F1-score
# No need for the complex comprehension here, pass all_reports directly
plot_overall_metrics(all_reports, 'f1-score', "Macro Avg F1-score for Classifiers on MNIST Datasets")

# Plot Overall Accuracy
# No need for the complex comprehension here, pass all_reports directly
plot_overall_metrics(all_reports, 'accuracy', "Overall Accuracy for Classifiers on MNIST Datasets")

```

```

import numpy as np

def calculate_mutual_information(counts_table):
    """
    Calculates mutual information from a 2x2 contingency table.
    counts_table = [[N_11, N_10], [N_01, N_00]]
    """
    N = np.sum(counts_table)

    # Joint probabilities
    P_xy = counts_table / N

    # Marginal probabilities
    P_x = np.sum(counts_table, axis=1) / N # P(S=1), P(S=0)
    P_y = np.sum(counts_table, axis=0) / N # P(K=1), P(K=0)

    mi = 0.0
    for i in range(2):
        for j in range(2):
            if P_xy[i, j] > 0:
                mi += P_xy[i, j] * np.log2(P_xy[i, j] / (P_x[i] * P_y[j]))
    return mi

```

```

import numpy as np

def calculate_mutual_information(counts_table):
    """
    Calculates mutual information from a 2x2 contingency table.
    counts_table = [[N_11, N_10], [N_01, N_00]]
    """
    N = np.sum(counts_table)

    # Joint probabilities
    P_xy = counts_table / N

    # Marginal probabilities
    P_x = np.sum(counts_table, axis=1) / N # P(S=1), P(S=0)
    P_y = np.sum(counts_table, axis=0) / N # P(K=1), P(K=0)

    mi = 0.0
    for i in range(2):
        for j in range(2):
            if P_xy[i, j] > 0:
                mi += P_xy[i, j] * np.log2(P_xy[i, j] / (P_x[i] * P_y[j]))
    return mi

```

```

# Cell 1: Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
✓ 1.8s

# Cell 2: Configuration and Setup
# Set the random seed for reproducibility as specified in the question
RANDOM_SEED = 6740
np.random.seed(RANDOM_SEED)

# Define parameters for the two normal distributions
# f0 = N(mu0, sigma0^2)
mu0 = 0
sigma0 = 1.0 # Standard deviation for f0 (variance is 1)

# f1 = N(mu1, sigma1^2)
mu1 = 0
sigma1 = np.sqrt(1.3) # Standard deviation for f1 (variance is 1.3)

# Define the number of samples from each distribution
n0 = 100 # Number of samples from f0 (pre-change)
n1 = 50 # Number of samples from f1 (post-change)
n_total = n0 + n1 # Total number of samples

print(f"Random seed set to: {RANDOM_SEED}")
print(f"Distribution f0: Normal(mean={mu0}, variance={sigma0**2:.1f})")
print(f"Distribution f1: Normal(mean={mu1}, variance={sigma1**2:.1f})")
print(f"Total samples: {n_total} (first {n0} from f0, next {n1} from f1)")

```

```

# Cell 3: Data Generation
# Generate samples from f0
data_f0 = np.random.normal(mu0, sigma0, n0)

# Generate samples from f1
data_f1 = np.random.normal(mu1, sigma1, n1)

# Concatenate the data to form the full sequence
x_sequence = np.concatenate((data_f0, data_f1))

print("\nData sequence generated.")
print(f"First 5 samples: {x_sequence[:5].round(2)}")
print(f"Samples around change point (98-102): {x_sequence[98:103].round(2)}")
print(f"Last 5 samples: {x_sequence[-5:].round(2)}")

```

```

# Calculate the constant part:
constant_term = -0.5 * np.log(1.3) # Using natural log as np.log is natural log
coefficient_x_squared = (3 / 26)

log_likelihood_ratios = constant_term + coefficient_x_squared * (x_sequence**2)

print("\nLog-likelihood ratios calculated.")
print(f"Constant term in l_k: {constant_term:.4f}")
print(f"Coefficient for x^2 in l_k: {coefficient_x_squared:.4f}")
print(f"First 5 l_k values: {log_likelihood_ratios[:5].round(4)}")
print(f"l_k values around change point (98-102): {log_likelihood_ratios[98:103].round(4)}")
print(f"Last 5 l_k values: {log_likelihood_ratios[-5:].round(4)}")

```

```

# Cell 5: Calculate CUSUM Statistics (S_k)
# S_k = max(0, S_{k-1} + l_k), with S_0 = 0

cusum_statistics = np.zeros(n_total + 1) # Initialize S_0 to S_{n_total}. S_0 is at index 0.

for k in range(n_total):
    # The current S_k is stored at index k+1 in the array
    # because S_0 is at index 0, S_1 at index 1, etc.
    cusum_statistics[k+1] = max(0, cusum_statistics[k] + log_likelihood_ratios[k])

print("\nCUSUM statistics calculation complete.")
print(f"S_0: {cusum_statistics[0]:.4f}")
print(f"S_{n0-1}: {cusum_statistics[n0-1]:.4f}")
print(f"S_{n0}: {cusum_statistics[n0]:.4f}") # This is S_k before processing x_k
print(f"S_{n0+1}: {cusum_statistics[n0+1]:.4f}") # This is S_k after processing x_{n0} (the first post-change observation)
print(f"S_{n_total}: {cusum_statistics[n_total]:.4f}")

```

```

# Cell 6: Train and evaluate SVM (Linear and Kernel RBF)
def train_and_evaluate_svm(xtrain, ytrain, xtest, ytest, dataset_name):
    print(f"\n--- Training SVM on {dataset_name} (Downsampled Training Data for SVC) ---")
    # Randomly downsample training data to m=5000 for efficiency.
    m = 5000
    if xtrain.shape[0] > m:
        sample_indices = np.random.choice(xtrain.shape[0], m, replace=False)
        xtrain_sampled = xtrain[sample_indices]
        ytrain_sampled = ytrain[sample_indices]
    else:
        xtrain_sampled = xtrain # Use full data if smaller than m
        ytrain_sampled = ytrain

    print(f"Using {xtrain_sampled.shape[0]} samples for SVM training.")
    # Linear SVM
    print(f"\n--- Training Linear SVM on {dataset_name} ---")
    # dual=False is recommended for n_samples > n_features for 'linear' kernel and 'l2' penalty
    svm_linear = SVC(kernel='linear', random_state=random_state, verbose=False)
    svm_linear.fit(xtrain_sampled, ytrain_sampled)
    y_pred_linear = svm_linear.predict(xtest)
    report_linear = classification_report(ytest, y_pred_linear, output_dict=True, zero_division=0)
    print(f"Classification Report for Linear SVM on {dataset_name}:\n")
    print(pd.DataFrame(report_linear).transpose())

    # Kernel SVM (RBF)
    print(f"\n--- Training Kernel SVM (RBF) on {dataset_name} ---")
    svm_rbf = SVC(kernel='rbf', random_state=random_state, verbose=False)
    svm_rbf.fit(xtrain_sampled, ytrain_sampled)
    y_pred_rbf = svm_rbf.predict(xtest)
    report_rbf = classification_report(ytest, y_pred_rbf, output_dict=True, zero_division=0)
    print(f"Classification Report for Kernel SVM (RBF) on {dataset_name}:\n")
    print(pd.DataFrame(report_rbf).transpose())

    return report_linear, report_rbf

svm_linear_digits_report, svm_rbf_digits_report = train_and_evaluate_svm(xtrain_digits, ytrain_digits, xtest_digits, ytest_digits, "MNIST Digits")
svm_linear_fashion_report, svm_rbf_fashion_report = train_and_evaluate_svm(xtrain_fashion, ytrain_fashion, xtest_fashion, ytest_fashion, "MNIST Fashion")

```